Build Your Own AI · Grades 7–12

No-code AI for Young Cambodian Learners

A hands-on guide to building AI with Google Teachable Machine



Rina Buoy, PhD

RinAI Academy
academy.rinabuoy.com

Draft edition

DRAFT

No-code AI for Young Cambodian Learners

© Rina Buoy, PhD. Draft edition.

Course website: `https://academy.rinabuoy.com`

Cover illustration and author photo: courtesy of the author.

Disclaimer. This book was prepared with the assistance of Claude Code, an AI tool from Anthropic. The author provides the content and reviews and revises all of the outputs.

Contents

1 What Is Artificial Intelligence (AI), Really? 1

1.1 AI is already around you . . . 1
1.2 What is AI? . . . 2
1.3 Two ways to "program" a computer . . . 2
1.4 Supervised learning: sorting candy . . . 3
1.5 More than pictures: other kinds of data . . . 5
1.6 Meet Teachable Machine . . . 5
1.7 Hands-on: your first look at Teachable Machine . . . 6

2 Image Classification 9

2.1 What is an image? . . . 9
2.2 What is image classification? . . . 10
2.3 Collecting data . . . 12
2.4 Training and testing with Teachable Machine . . . 13
2.5 Hard test cases . . . 13
2.6 Practice time . . . 13

3 Image Classification Project Lab: Train, Tune, Save and Share 15

3.1 Recap . . . 15
3.2 What training does . . . 16
3.3 Three questions about training . . . 16
3.4 The tuning loop . . . 18
3.5 Train-test split . . . 19
3.6 Saving your project . . . 20
3.7 Sharing your project . . . 20
3.8 Your turn: project lab . . . 21

A How Computers See Pictures 22

A.1 A picture is a grid . . . 22
A.2 Pixels are numbers . . . 22

DRAFT

A.3 How many numbers is a photo? . . . 23
A.4 Why this matters for your projects . . . 23
A.5 Try it . . . 23

B Sound, Waves and Spectrograms **24**
B.1 What is sound? . . . 24
B.2 Reading a waveform . . . 24
B.3 From sound to numbers . . . 25
B.4 Why a waveform is not enough . . . 25
B.5 The spectrogram . . . 25
B.6 Why spectrograms help AI . . . 25
B.7 Try it . . . 26

C How a Model Learns **27**
C.1 Learning by guessing and correcting . . . 27
C.2 What gets adjusted? . . . 27
C.3 The three settings . . . 27
C.4 Accuracy and error: right or wrong? . . . 28
C.5 Why accuracy levels off . . . 28
C.6 Overfitting and underfitting . . . 28
C.7 What is a neural network? . . . 29
C.8 Try it . . . 29

D Pre-trained Models and Keypoints **30**
D.1 Why you need so few samples . . . 30
D.2 Who does what . . . 30
D.3 What is a keypoint? . . . 31
D.4 Why poses are easier than pictures . . . 31
D.5 Where pose models can go wrong . . . 31
D.6 Try it . . . 32

E Measuring Results **33**
E.1 Accuracy . . . 33
E.2 Why one number can hide problems . . . 33
E.3 The confusion matrix . . . 33

DRAFT

E.4 Confidence 34
E.5 Fair tests 34
E.6 Try it 35

About the Author **36**

DRAFT

DRAFT

Chapter 1

What Is Artificial Intelligence (AI), Really?

At a glance

You will see how much of your day already runs on AI, learn what AI actually is, and meet the idea that powers every project in this book: a computer can *learn from examples* instead of being told exact rules. You finish by opening Google Teachable Machine, the free, no-code tool you will use all the way through.

Learning objectives

By the end of this chapter you can:

1. Give everyday examples of AI and explain what each one learns.
2. Explain the difference between giving a computer rules and showing it examples.
3. Describe supervised learning as *data* → *labels* → *model* → *prediction*.
4. Name the three kinds of project in Teachable Machine (Image, Audio, Pose) and say what each one recognizes.
5. Open a blank Image Project and find the Class boxes, the webcam button, the Train Model button and the Preview panel.

Key terms

Artificial intelligence (AI) · **model** · **training** · **data** · **label** · **prediction** · **supervised learning** · **pattern** · **Teachable Machine** · **class**

1.1 AI is already around you

Think about your day so far. Did you use a face filter on Instagram or Facebook? Watch a "For You" feed on TikTok or YouTube? Ask a voice assistant such as Siri (iPhone) or Google Assistant (Android) a question, listen to music suggested by Spotify or YouTube Music, unlock your phone with Face ID, or let autocorrect fix a typing mistake?

Every one of these runs on AI. Each also *learned* something to do its job:

DRAFT

What you used	What it learned
Face filters	what a face looks like
Recommendations	what you like to watch
Voice assistants	to understand speech
Spam filters	what spam looks like
Music discovery	your taste in music
Autocorrect	how you usually type

Activity (icebreaker). Tick the ones you have used today. Which surprised you? The rest of this chapter explains how such systems learn.

1.2 What is AI?

Artificial Intelligence (AI) means making a computer program (or a *model*) as smart as a human at some task. Take a simple question: *is this person sad or happy by looking at a face image?* You can tell at a glance. Does a computer know? Not unless it has been taught.

Figure 1.1 shows the question again: is this person sad or happy?



Figure 1.1. Emotion recognition: can you tell at a glance whether this person is sad or happy? A computer has to be taught.

Note. Recently, artificial intelligence has also been called **superintelligence**, especially when people talk about AI that could become smarter than humans rather than only as smart as a human at one task. This is the instructor's note and is not on the Week 1 slides.

1.3 Two ways to "program" a computer

Way 1: step-by-step instructions. A person writes exact rules and the computer follows them precisely. The same rules always give the same result. But the program breaks on cases

nobody wrote a rule for. For example, a rule for smiling is if the mouth is 30% wider than normal.

Figure 1.2 shows the smile rule in action: the mouth is 30% wider than normal, so the rule says "smile detected".

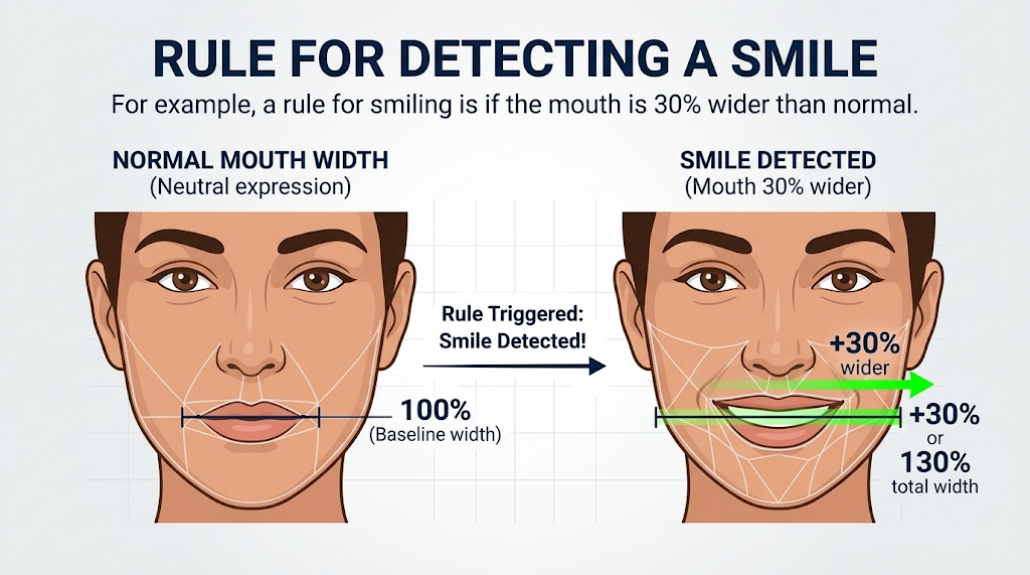


Figure 1.2. A rule for detecting a smile: if the mouth is 30% wider than normal, the rule says

Way 2: AI. A person shows lots of *examples*. The computer finds the pattern itself. It "learns" instead of being told, and it can handle new, never-seen examples. Such modelling approaches are data-driven. For example, we give the computer program (aka an AI model) 1000 images of sad faces and another 1000 images of smiling faces, and let the computer program figure what makes a smiling face different a sad one.

In Figure 1.3 the model receives 1,000 sad faces and 1,000 smiling faces and works out the differences by itself.

Think about it. Can a computer learn something without anyone telling it exactly what to do? Discuss with a partner for 60 seconds before reading on.

Why not just write rules? Imagine writing rules for "smiling". How wide must the mouth curve? What about different faces, lighting, and camera angles? There are far too many cases. Showing examples is much easier, and it is how Teachable Machine works.

1.4 Supervised learning: sorting candy

Imagine you hand a friend 100 candies already sorted into a "red" pile and a "blue" pile. After a while, they start sorting *new* candies correctly on their own. That is **supervised learning**. It has four steps:

1. **Data.** Show lots of labeled examples.
2. **Labels.** Each example comes with the correct answer.
3. **Model.** The AI studies the patterns.

DRAFT

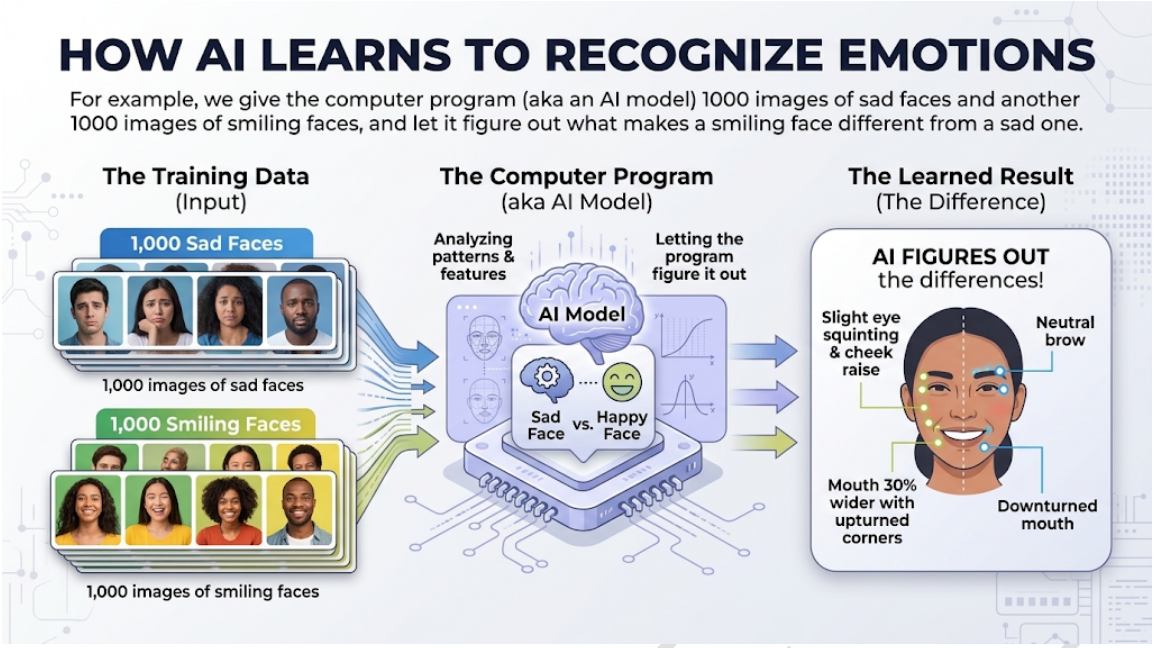


Figure 1.3. How AI learns to recognise emotions: 1,000 sad faces and 1,000 smiling faces go into the model, which figures out the differences by itself.

4. **Prediction.** It guesses labels for new data it has never seen.

What does "supervised" mean? To *supervise* is to watch over and guide, like a teacher checking homework. In supervised learning, a person (the supervisor) gives the computer the **correct answer, called a label,** for every training example. The model learns by comparing its guesses with those answers. If the examples came with no answers, the learning would be *unsupervised*. This definition is the author's explanation, not from the slides.

Figure 1.4 summarizes the four steps.

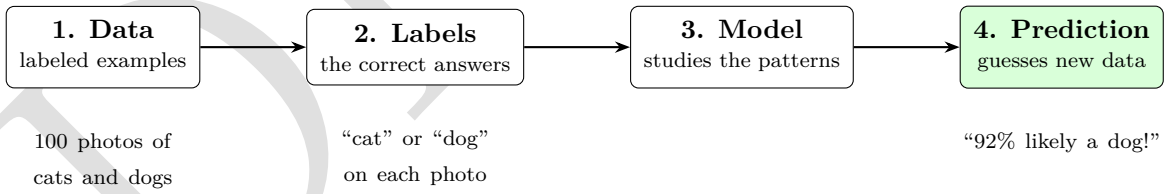


Figure 1.4. Supervised learning in four steps: data, labels, model, prediction.

Worked example 1: cat or dog. Show the model 100 photos labeled "cat" and 100 labeled "dog". It studies the differences, such as shapes, ears and patterns. Then you show it a brand-new photo. It answers with a *confidence*: "That's 92% likely a dog!"

Worked example 2: a smile detector.

- *Training*: give the computer faces that are smiling and faces that are not smiling, and let it search for what makes them different.
- *Result*: a **trained AI model**.
- *Prediction*: show it a new face and ask, "Smiling?"

The whole flow is: **data → training (searching for patterns) → trained model → prediction.**

Figure 1.5 applies the four steps to the smile detector.

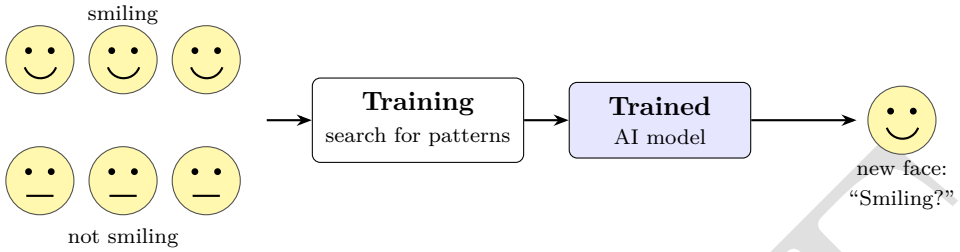


Figure 1.5. The smile detector: examples of smiling and not-smiling faces train a model, which then answers

1.5 More than pictures: other kinds of data

The same idea works for any data (technically called “modality”) a computer can read:

- **Image**: an emotion detector that looks at faces.
- **Audio**: an emotion detector that listens to voices (happy, angry, surprised).
- **Pose**: a bad-vs-good posture detector that tracks the skeleton of a sitting person.
- **Text**: a polite-vs-impolite detector, or sentiment analysis that labels a review positive, neutral or negative.

The limit is your imagination.

Figure 1.6 shows that audio, image and pose all follow the same path from examples to a trained model.

1.6 Meet Teachable Machine

Google Teachable Machine (teachablemachine.withgoogle.com) is a free tool that lets *you* train a real AI model in minutes, right in your browser. Extra course materials and helper tools for Teachable Machine are collected on the course website.[1]

- **No coding required**: point, click and train.
- **Fast results**: a working model in minutes.
- **Uses your webcam and microphone**: you collect your own real data live.

It offers three ways to train an AI:

[1] Course web app: `https://seqmodelling.rinabuoy.com/teachable-machine/`

DRAFT

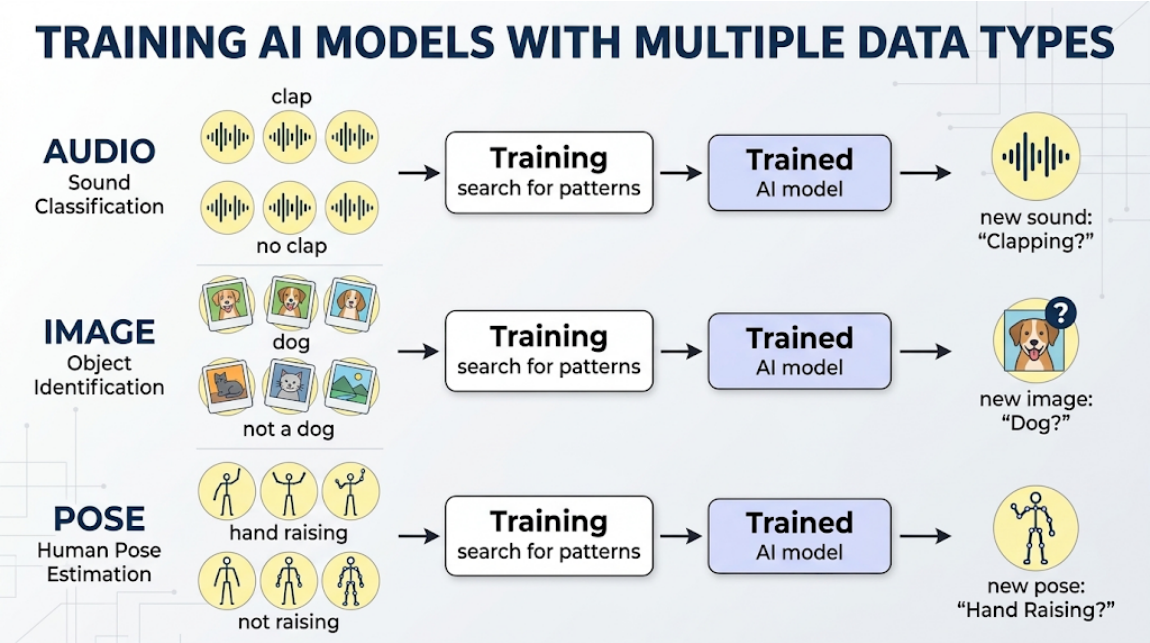


Figure 1.6. Training AI models with multiple data types: audio (clap / no clap), image (dog / not a dog) and pose (hand raising / not raising) all follow the same path: examples, training, a trained model, then a prediction for new data.

Project type	Recognizes	Example
Image	pictures from your webcam	rock-paper-scissors detector
Audio	sounds and short clips	clap or whistle detector
Pose	body position and movement	exercise rep counter

Figure 1.7 shows the three project types side by side.



Figure 1.7. The three kinds of project in Teachable Machine: Image, Audio and Pose.

Discuss. If you built an AI for your favourite hobby, would it need to *see*, *hear*, or track *movement*? Pick your hobby and your AI type.

1.7 Hands-on: your first look at Teachable Machine

Activity 1: create your workspace.

1. Go to teachablemachine.withgoogle.com.
2. Click **Get Started**. No account or sign-up is required to try it.
3. When the browser asks, allow camera and microphone access.

Activity 2: explore real example projects. Open two or three public example projects and try them live. Write down: What classes (categories) did the project use? How accurate did it seem when you tested it live? What surprised you, or fooled the model? *Tip:* try to trick the model on purpose. That is how you learn its limits.

Activity 3: open your first Image Project. Open a blank Image Project. Do **not** train anything yet. Just find:

- the **Class boxes** where you will add examples,
- the **webcam capture button**,
- the **Train Model** button,
- the live **Preview** panel on the right.

Figure 1.8 is a simplified sketch of the workspace you should find.

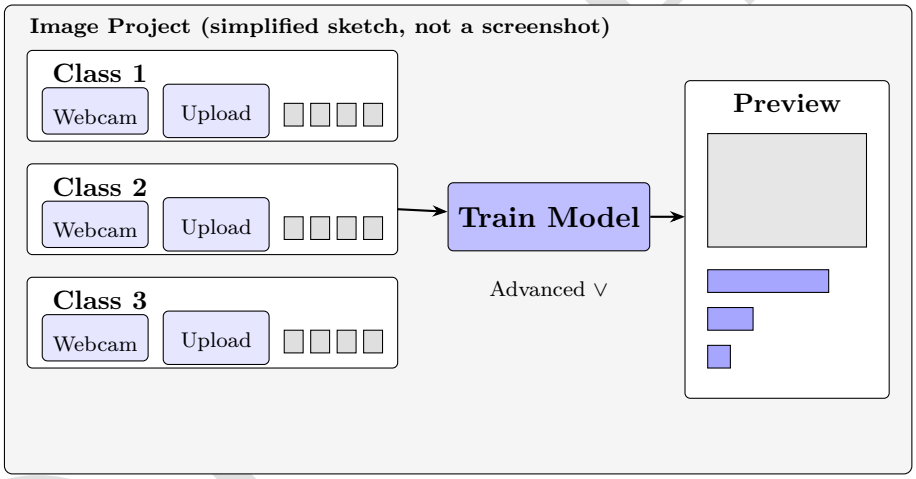


Figure 1.8. A simplified sketch of an Image Project workspace: class boxes with Webcam and Upload buttons, the Train Model button, and the live Preview panel.

Activity 4: test previous models. Try out two image models that were trained earlier. Open each link, allow the camera if asked, and test the model live:

- **Mask detector** (classes: *Mask*, *No Mask*, *Incorrect*): https://teachablemachine.withgoogle.com/models/
- **Rock-paper-scissors** (classes: *paper*, *scissor*, *rock*): https://teachablemachine.withgoogle.com/models/s

For each model:

1. Test every class and write down how confident the model is.
2. Try to fool it. For the mask model, what happens with a hand over the mouth or a mask worn under the nose? For the rock-paper-scissors model, what happens with a different hand position, distance or lighting, or with no hand at all?
3. Which class is the hardest for it? What kinds of examples might have been missing from its training data?

DRAFT

Exit ticket. Sketch an idea for a 2–3 class image classifier that you will build next week. Ideas: hand gestures, fruit vs. vegetable, facial expressions, school objects.

Summary

- AI is all around you, and each system learned its skill from examples.
- Rules break on cases nobody foresaw; learning from examples handles new cases.
- Supervised learning = data → labels → model → prediction; the model answers with a confidence.
- The same idea works for images, sounds, poses and text.
- Teachable Machine lets you train Image, Audio and Pose models without writing code.

Check your understanding

1. Name three apps you used this week and what each one learned.
2. In your own words, what is the difference between a rule and an example?
3. In the smile detector, what are the *data*, the *labels* and the *prediction*?
4. Which Teachable Machine project type would you use for a clap detector? For a posture checker?

Answer hints: (3) data = face photos; labels = smiling / not smiling; prediction = the model's guess for a new face. (4) Audio; Pose.

Further practice

Bring your classifier sketch to the next chapter. Write down the 2–3 classes you will use and where you could collect the pictures.

DRAFT

Chapter 2

Image Classification

At a glance

Before a computer can tell a can from a plastic bottle, you must understand what an image is and how to build a good set of examples. This chapter explains images, classification, and how to collect, store and test pictures, using a trash classifier as the running example.

Learning objectives

By the end of this chapter you can:

1. Describe what an image is and name the common image types.
2. Explain image classification and tell binary from multi-class problems.
3. Identify the *input* and *output* of an image classifier.
4. Collect, diversify and store a dataset with one folder per class.
5. Describe how training data, a trained model and test data fit together.

Key terms

Image · **RGB** · **grayscale** · **binary image** · **width and height** · **image classification** · **binary classification** · **multi-class classification** · **input** · **output (label)** · **dataset** · **diversity** · **training data** · **test data**

2.1 What is an image?

A digital **image** is a grid of tiny dots called pixels. Several types appear in this course and in real life:

- **RGB** images store **R**ed, **G**reen and **B**lue values for every pixel, giving full colour.
- **Grayscale** images keep only light and dark.
- **Binary** images have just two values, black and white.
- Special images include **X-rays** (medical scans of the chest, for example) and **satellite** photos (a city seen from above).

Figure 2.1 shows the same flower as an RGB, a grayscale and a binary image.

Image size is described by **width** and **height**. Images can be small, medium or large. A

DRAFT

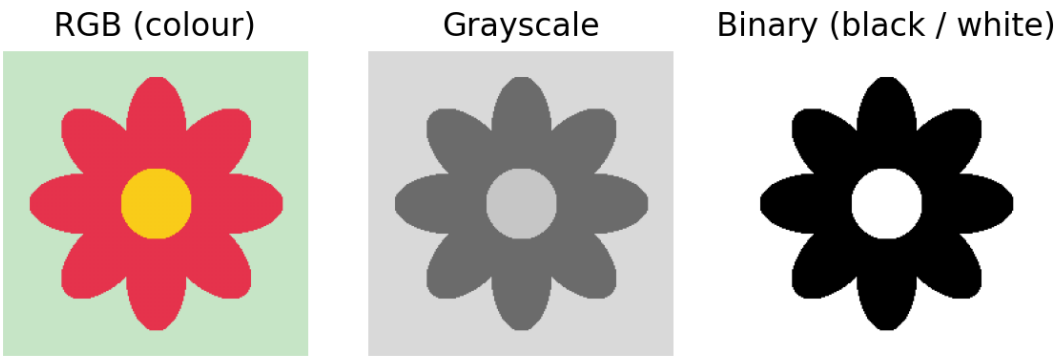


Figure 2.1. The same flower as an RGB (colour), grayscale and binary image.

bigger image holds more detail but takes more computing power. (In Chapter 7 you will see that Teachable Machine resizes every webcam frame to 224 × 224 pixels.)

Go deeper. Want to know what a computer really stores for each pixel? See Appendix A, *How Computers See Pictures*.

2.2 What is image classification?

Image classification means giving an image a label. In this chapter the labels are kinds of rubbish:

- **Binary classification**: only two possible labels, for example *can (metal)* vs. *plastic*.
- **Multi-class classification**: three or more labels, for example *can (metal)* vs. *plastic* vs. *paper*.

Figure 2.2 compares the two: with two labels the model must pick one or the other, while with three or more labels it picks one of several.

The elements of a classifier

Every image classifier has the same two parts. For a **trash image classifier**:

- **Input**: an image.
- **Output**: a label (Plastic, Can or Paper).

Figure 2.3 shows the input and output of the trash classifier.

Why classify images?

Sorting rubbish for recycling is one reason; the same idea reaches medicine (X-rays), farming, traffic signs and satellites. **Think about it:** list other cases of image classification in your own community.

DRAFT

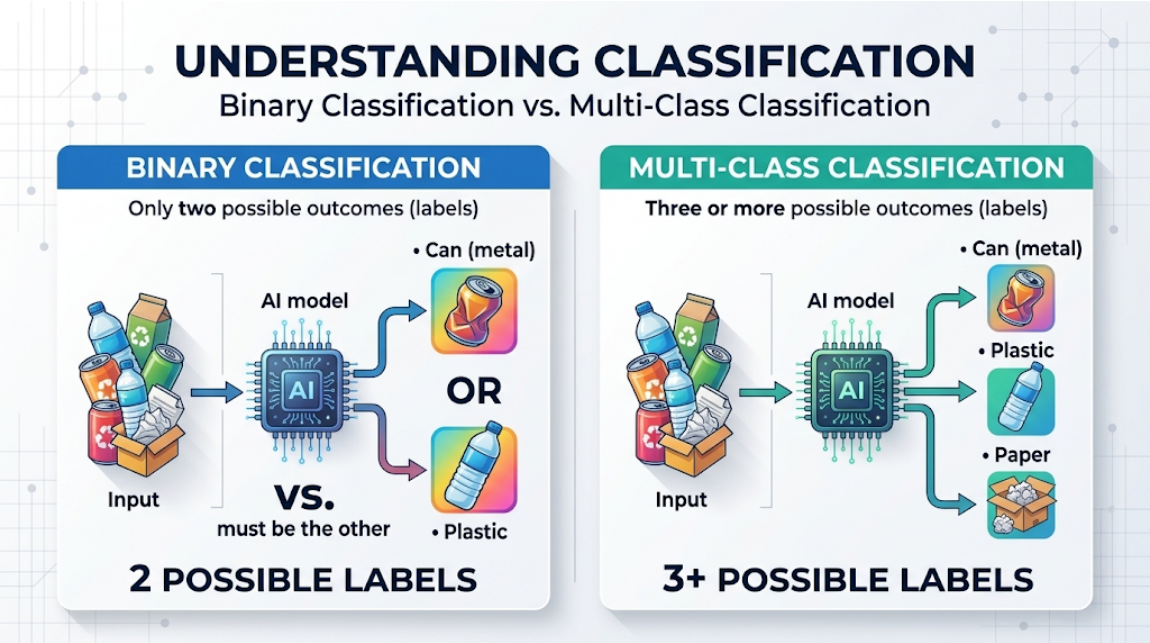


Figure 2.2. Understanding classification: binary classification has two possible labels (can or plastic); multi-class classification has three or more (can, plastic or paper).

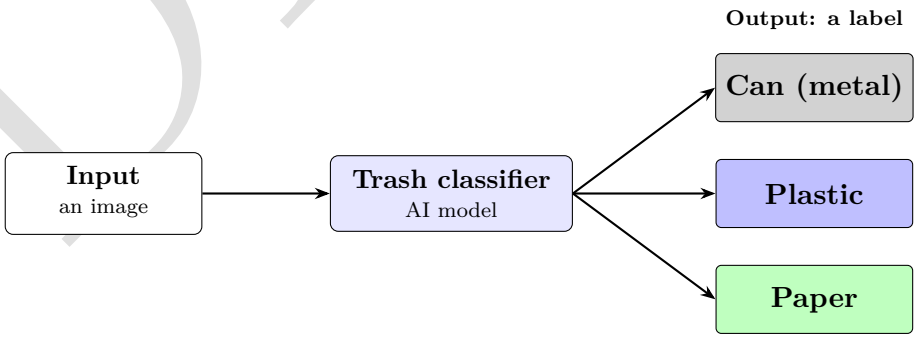


Figure 2.3. A trash image classifier: an image goes in and a label comes out.

DRAFT

2.3 Collecting data

A model can only learn from what you show it, so data collection is the most important step.

Where do pictures come from?

- a **camera**,
- a **mobile phone**,
- a **webcam**,
- **Google Image Search**.

Diversity of data

Do not take ten photos of the same can. Collect *different* cans, *different* plastics and *different* papers. Vary the angle, the light and the background. If every can in your set is the same brand, the model may only learn that brand.

Figure 2.4 contrasts the two approaches: ten photos of the same can teach the model only one brand, while different items, angles, lighting and backgrounds teach it what a can, a plastic bottle and paper really look like.

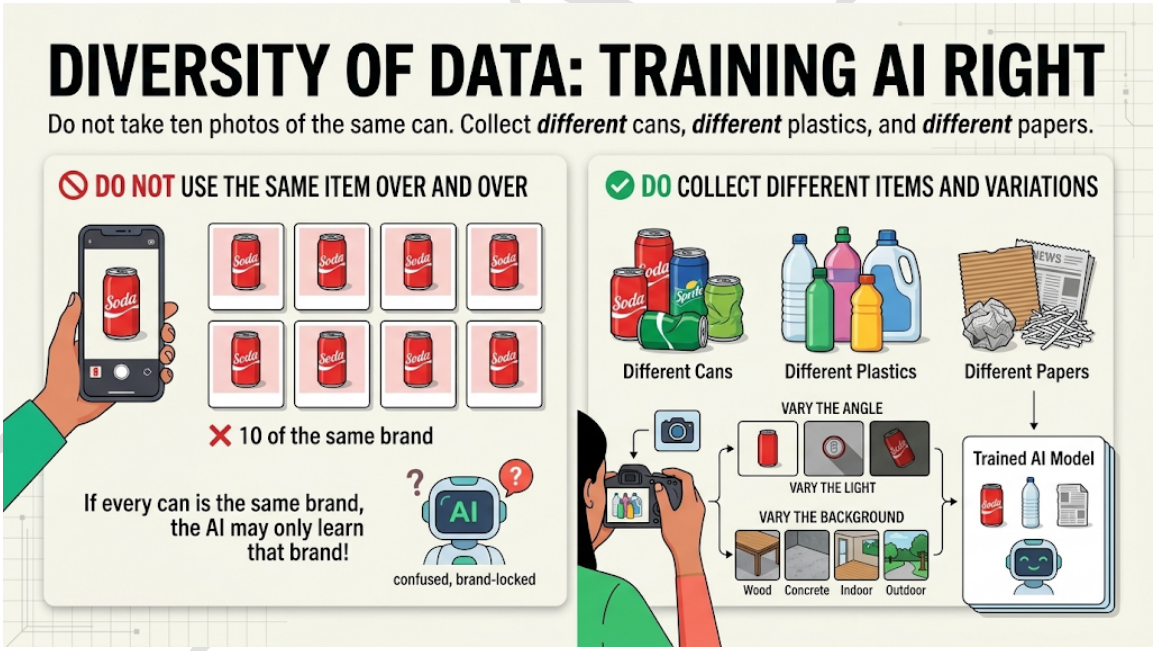


Figure 2.4. Diversity of data: do not use the same item over and over; collect different items and vary the angle, the light and the background.

Storing your data

Keep one **folder per class**:

- `Can (metal)` folder

- `Plastic` folder
- `Paper` folder

Figure 2.5 shows how one folder per class keeps the dataset organized.

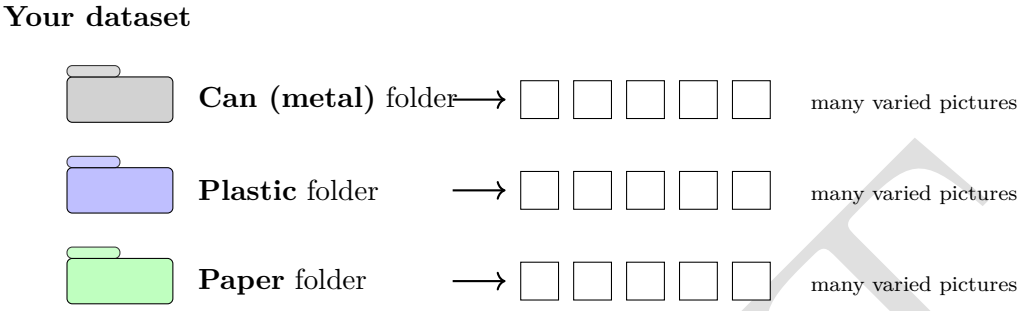


Figure 2.5. One folder per class keeps the dataset organized.

Putting a picture in the wrong folder gives the model a wrong lesson, so check carefully.

2.4 Training and testing with Teachable Machine

1. **Training data.** The folders of pictures go into Google Teachable Machine. Each folder becomes a class.
2. **Training.** Teachable Machine learns from the folders and produces the **trash classifier model**.
3. **Test data.** Show the model *new* pictures it has not seen. It answers CAN, PAPER or PLASTIC.

Always test with pictures that were not in the training folders, otherwise you are only checking its memory, not its learning.

A trained model for reference. A finished version of the trash classifier is online. Open it, allow the camera if asked, and test it with items from your own bag or classroom: https://teachablemachine.withgo
It is an Image model with three classes, *Plastic*, *Metal* and *Paper*. (Its "Metal" class plays the role of the "Can" class in this chapter.) Try the hard cases from Section 2.5 and see where it fails.

2.5 Hard test cases

Real life is messier than your training set. Good test pictures include a **dented can**, a **newspaper advertisement page** (paper, but colourful and full of text) and a **clear plastic item against a wall**. If the model struggles with these, add more varied examples of that class and train again. (Chapter 3 shows how.)

2.6 Practice time

Build the trash classifier, or the classifier you sketched in Chapter 1:

DRAFT

1. Decide on 2–3 classes.
2. Collect a varied set of pictures for each class.
3. Store each class in its own folder (or add them directly in Teachable Machine).
4. Train, then test with new pictures, including a few hard ones.

Summary

- An image is a grid of pixels; common types are RGB, grayscale and binary.
- Classification gives an image a label; it is binary with two labels and multi-class with three or more.
- A classifier takes an image as input and outputs a label.
- Good data is varied, correctly labeled and stored one folder per class.
- Always test with pictures the model has never seen.

Check your understanding

1. Is “cat / dog / rabbit” binary or multi-class? Why?
2. Name three ways to get pictures for a dataset.
3. Why is it a bad idea to take all your “can” photos of one brand?
4. Why should test pictures be different from training pictures?

Answer hints: (1) multi-class (three labels). (3) The model may learn the brand, not “can”. (4) To measure learning, not memory.

Further practice

Find a classification problem in your own community that could be solved with pictures. Write its input, its output labels and where you would collect the data.

DRAFT

Chapter 3

Image Classification Project Lab: Train, Tune, Save and Share

At a glance

You have a dataset; now you will train a model, understand the three settings that control training, tune them by trying, and finally save and share your project. The lab ends with a project you present to friends.

Learning objectives

By the end of this chapter you can:

1. Explain what happens during training and recognize an untrained vs. a trained model.
2. Describe **epochs**, **learning rate** and **batch size**, and what goes wrong when each is too low or too high.
3. Use a train → test → retry loop to improve a model, and explain why you test with new pictures.
4. Split your pictures into a training set and a test set.
5. Save a Teachable Machine project and share a model with a link.
6. Build and share a rock-paper-scissors or mask/no-mask classifier.

Key terms

Training · **epoch** · **learning rate** · **batch size** · **overfitting (memorizing)** · **training set** · **test set** · **train-test split** · **Advanced settings** · **Preview** · **export** · **shareable link**

3.1 Recap

Last chapter you met image types (RGB, grayscale, binary), multi-class classification (can vs. plastic vs. paper), and the idea of **one folder per class** for storing examples. We now put those folders to work.

DRAFT

3.2 What training does

In Teachable Machine each class holds example images. The workspace shows, for example, three classes (**Can**, **Plastic**, **Paper**) with ten image samples each, and a **Train Model** button.

Before training, the model is guessing. Show it a picture and the Output bars are close to an even split, roughly a third for each class (the course screenshots show about 31–36% each). **After training**, the same kind of picture gets a confident answer. In the course examples a crushed red can went from about 31/38/31% to **100% Can**, a white plastic bottle neck scored **99% Plastic**, and crumpled paper scored **92% Paper**.

Training means the model looks at your examples over and over, getting better at *predicting the right output*.

Figure 3.1 shows example output bars before and after training.

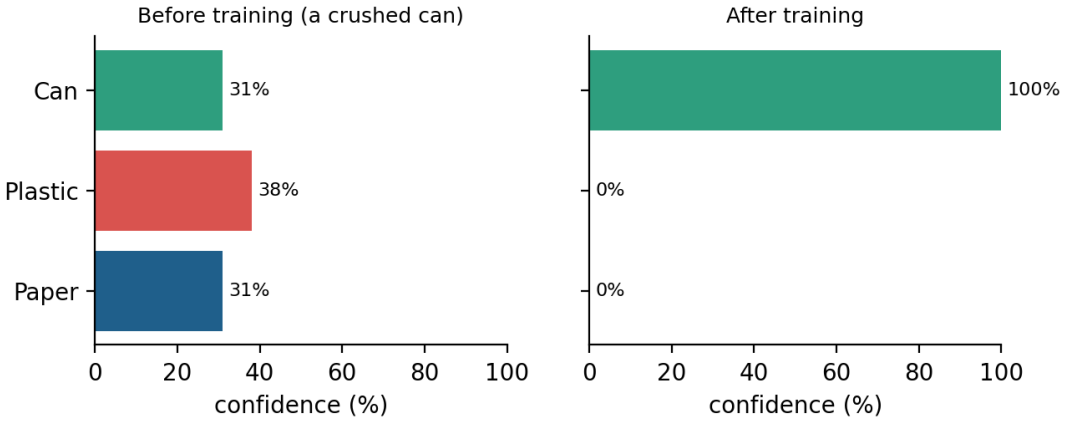


Figure 3.1. Output bars for a crushed-can picture before and after training (example values).

3.3 Three questions about training

Training is controlled by three settings in the **Advanced** panel:

Question	Setting	Teachable Machine default
How long to train?	**Epochs**	50
How large a step to take?	**Batch size**	16
How fast to learn?	**Learning rate**	0.001

A **Reset Defaults** button restores them, and **Under the hood** shows accuracy per class and a confusion matrix (Chapter 7).

Figure 3.2 shows the Advanced panel with its default values.

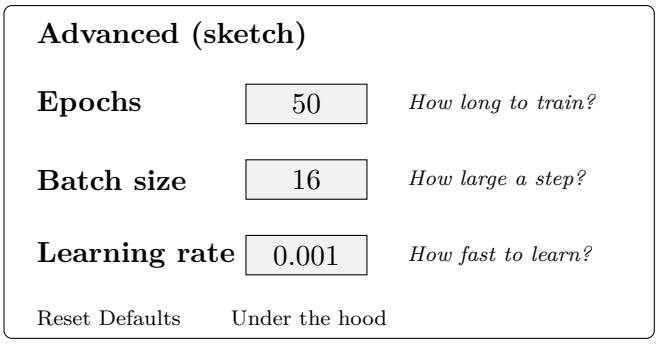


Figure 3.2. The Advanced panel with Teachable Machine's default values.

Epochs: how long to train

An *epoch* is one full pass over all your samples.

- **Too low (1–5):** not enough time to learn.
- **Too high (100–1000):** the model *memorizes* your pictures instead of understanding them. This is called overfitting.
- **Solution:** find a good value by trying many times (technically called "tuning").

Figure 3.3 illustrates what too few and too many epochs do.

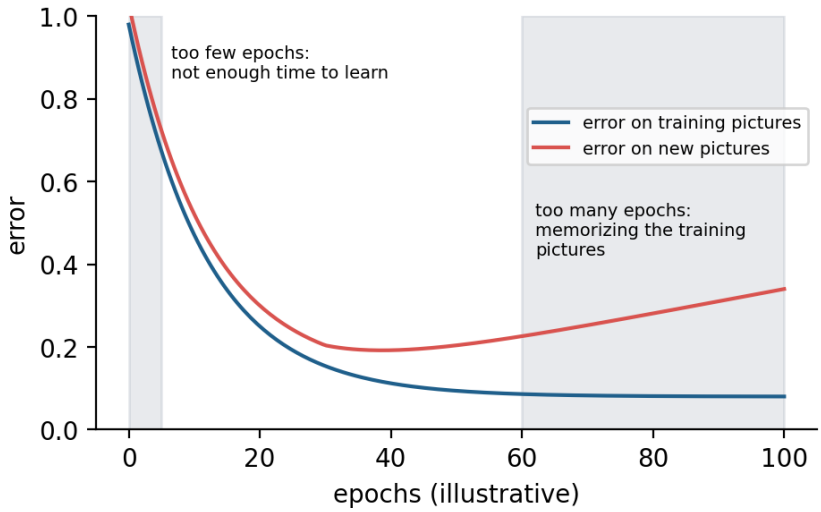


Figure 3.3. An illustration of the epochs idea: with too few epochs the model has not learned; with too many it memorizes the training pictures and does worse on new ones.

Learning rate: how fast to learn

The learning rate is how big each adjustment to the model is.

- **Too small** (for example 0.000001): learning crawls.
- **Too large** (for example 1): the model overshoots and never settles.
- **Solution:** again, try values and test.

Batch size: how many samples at a time

The batch size is how many samples the model looks at in each step. Examples shown are 1 and 10. The right choice depends on **your computer's computing power**.

Go deeper. Curious what happens inside the model during training? See Appendix C, *How a Model Learns*.

Teaching note. The slides picture hand-washing laundry next to a washing machine for this idea; read it as "a few items at a time versus many at once".

3.4 The tuning loop

No single setting is right for every project. Use this loop:

1. **Try some settings.**
2. **Test the model** with new pictures.
3. **Not OK?** Change a setting (or add better data) and go back to step 1.
4. **OK?** Done.

Figure 3.4 shows the tuning loop.

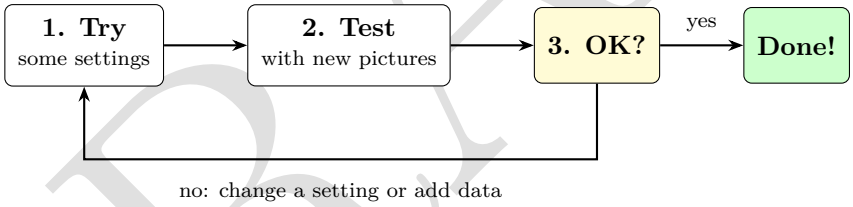


Figure 3.4. The tuning loop: try, test, and change a setting until the result is acceptable.

Why not test on the training pictures?

Testing with the same pictures the model trained on is like giving students 100 practice questions to prepare for an exam, and then putting exactly the same 100 questions in the exam. A student who scores 100% might truly understand the subject, or might simply have **memorized** the answers. From the scores alone, you cannot tell which.

A model is the same. Pictures it has already seen will almost always get a high score, even if the model has only memorized them (this is the overfitting from the epochs section). To find out whether it has really **learned** the idea of a can, a plastic bottle or a hand gesture, test it with **new pictures** it has never seen, like fresh exam questions. *(The exam comparison is the author's analogy.)*

Testing must use pictures the model has not seen. Compare which hard cases fail, such as a newspaper ad or a clear plastic item, and add varied examples of those classes.

DRAFT

3.5 Train-test split

Section 3.4 said to test with pictures the model has never seen. The easiest way to get such pictures is a **train-test split**: before you train, divide your pictures for each class into two groups.

- The **training set** is what the model learns from.
- The **test set** is kept aside. The model never sees it while learning, and you use it **only** to check the result, like the real exam.

A common choice is about **80% for training and 20% for testing**. (This is general practice, not from the slides.) For example, if you have 50 pictures of plastic bottles, put 40 in the training set and keep 10 for testing. Do the same for every class.

Figure 3.5 shows the split.

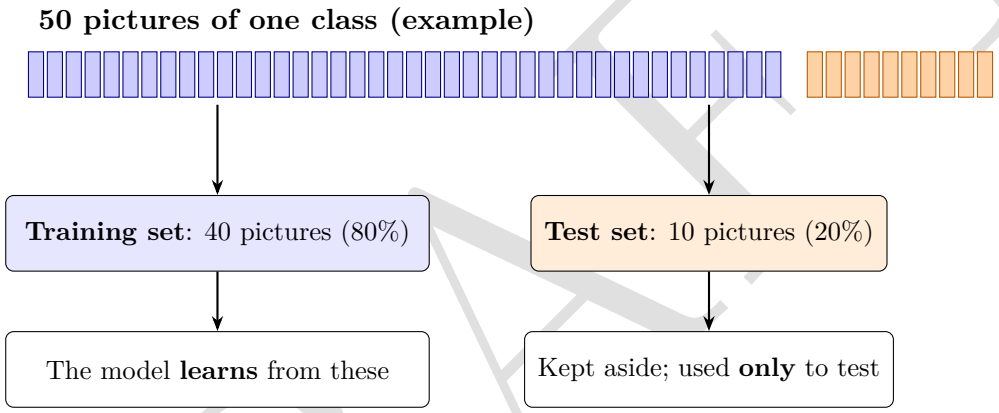


Figure 3.5. A train-test split: of 50 pictures of one class, 40 are used for training and 10 are kept aside for testing.

Rules for a fair split

1. **Split first, then train.** Set the test pictures aside *before* you press Train Model, and never add them to a class.
2. **Keep the test set untouched.** If a test picture fails, do not copy it into the training set and then re-test with the same picture; take new pictures for the next round of testing.
3. **Mix the split.** Do not use only the first photos you took for training and the last ones for testing. Pick at random, so both groups contain a mix of angles, light and backgrounds.
4. **Test every class.** Keep some test pictures for each class, in roughly equal numbers.
5. **Use other people if you can.** A friend's photos or handwriting make a tougher, more honest test than your own.

How to do it in Teachable Machine. Keep your test pictures in a separate folder on your computer and do not upload them to any class. After training, test by showing them to the Preview (webcam or *Upload*). Teachable Machine also reports accuracy on its own held-out samples: open **Advanced** → **Under the hood** (see Chapter 7). *(Author's note: Teachable Machine's built-in split is not described in the slides; check the exact percentages in the tool before quoting them.)*

DRAFT

Later chapters build on this idea: in Chapter 7 you test with labelled folders of fresh pictures, and in Chapters 8 and 9 you must never augment the test set.

3.6 Saving your project

Saving keeps your work for later. Teachable Machine lets you save **online** (to Google Drive, or as a shareable link) or **on your computer** (download the project file). Open the project menu (top left) to find *Save project to Drive* and *Download project as file*. A model link alone cannot be retrained, so save the project itself if you want to add more images.

3.7 Sharing your project

Sharing lets anyone try your model.

1. Click **Export Model**.
2. Choose **Upload (shareable link)**. The export panel also lists TensorFlow.js, TensorFlow and TensorFlow Lite options.
3. Copy the link (it looks like `teachablemachine.withgoogle.com/models/…`).

Anyone with the link can use the model on their own webcam, so share it with care.

Figure 3.6 shows the page that opens from a shared link: the model's web address and files on the left, and a live preview on the right where anyone can test the model (here with a picture of a newspaper advertisement, which the model labels Paper).

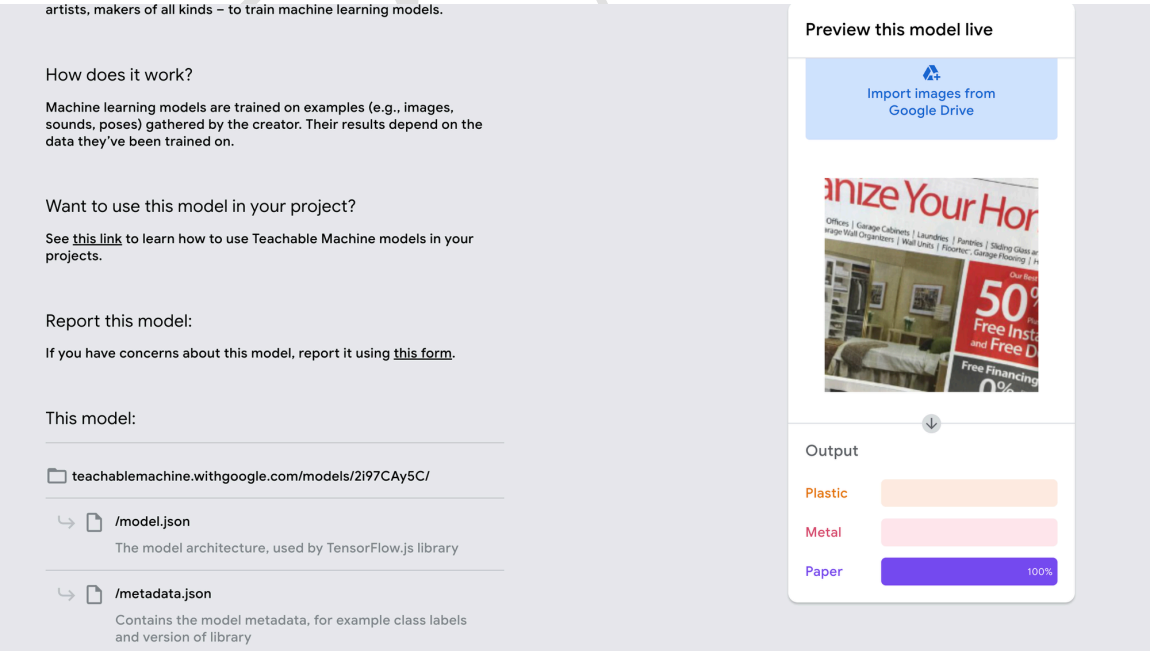


Figure 3.6. The page for a shared model: its link and files (left) and a live preview with Output bars (right).

3.8 Your turn: project lab

Train **either** a rock-paper-scissors **or** a mask / no-mask model:

1. Create 3 (or 2) classes and collect varied images for each. Keep about 20% of the pictures aside as a test set.
2. Train with the default settings and test.
3. Tune: try changing epochs, then learning rate or batch size, one at a time.
4. Test with your test set, then with new people and backgrounds.
5. Save the project and share your model link with your instructor or friends.

Summary

- Training makes the model better at predicting the right output; untrained outputs are close to even guesses.
- Epochs, learning rate and batch size control how long, how fast and how big a step; too much or too little of each causes problems.
- Improve by looping: try, test, adjust.
- Split your pictures into a training set (about 80%) and a test set (about 20%) before you train, and keep the test set untouched.
- Test with new pictures, never the training ones: like an exam that repeats the practice questions, a test on training pictures cannot tell understanding from memorizing.
- Save the project to keep working; share with an exported link.

Check your understanding

1. What do the Output bars look like before training?
2. What does a *very high* number of epochs risk?
3. Why change only one setting at a time?
4. What is the difference between saving a project and sharing a model link?

Answer hints: (1) near-equal bars. (2) Memorizing instead of understanding. (3) So you know which change caused the result. (4) A project can be retrained; a link only runs the model.

Further practice

Record the settings and accuracy for each experiment in a small table, then explain which setting made the biggest difference.

DRAFT

Appendix A

How Computers See Pictures

For curious readers. You do not need this appendix to finish the course. It explains, step by step, what Chapters 2 and 7 mean when they say "a picture is a grid of numbers".

A.1 A picture is a grid

Look closely at a photo on a screen and you would find tiny squares called **pixels** (short for "picture elements"). Each pixel is one dot of one colour. Put thousands of them in rows and columns and your eyes see a smooth picture.

A.2 Pixels are numbers

A computer cannot "look". It stores a number for each pixel.

- In a **grayscale** picture, one number per pixel says how dark it is. In Chapter 7, 0 means blank paper and 255 means the darkest ink.
- In an **RGB** (colour) picture, each pixel has **three** numbers: how much **R**ed, **G**reen and **B**lue light to mix. Mixing the three gives any colour.
- In a **binary** picture, each pixel is only one of two values, black or white.

Figure A.1 shows how a digit looks as a grid of numbers.

Here is a tiny example: a 5×5 grayscale picture of the letter **L**. The dark strokes have big numbers and the empty paper has zeros.

	col 1	**col 2**	**col 3**	**col 4**	**col 5**
row 1	255	0	0	0	0
row 2	255	0	0	0	0
row 3	255	0	0	0	0
row 4	255	0	0	0	0
row 5	255	255	255	255	255

Author's example, made up to illustrate the idea. If you squint at the table, the 255s draw the letter. That is all the computer has to work with.

DRAFT

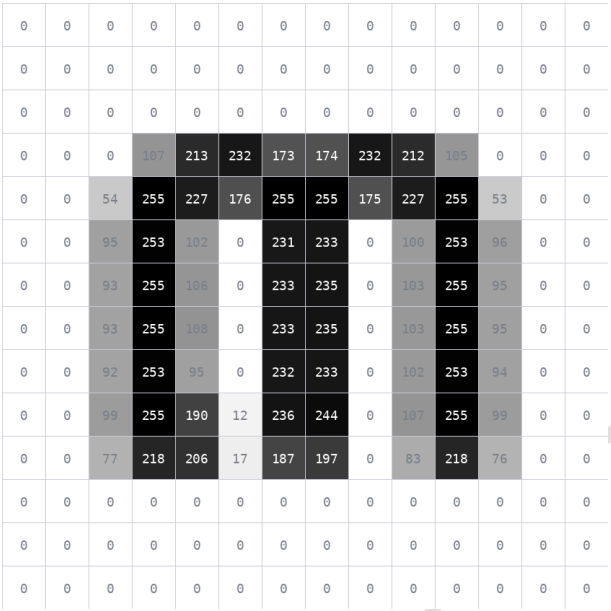


Figure A.1. A Khmer digit ო shrunk to a 14 × 14 pixel grid: each square holds a number from 0 (blank paper) to 255 (darkest ink). *Image from the course slides by Rina Buoy, PhD.*

A.3 How many numbers is a photo?

Teachable Machine resizes every camera frame to **224 × 224** pixels (Chapter 7). A colour frame has three numbers per pixel, so one picture is $224 \times 224 \times 3 = \mathbf{150{,}528}$ **numbers**. *(The multiplication is the author's arithmetic from the figures in Chapter 7.)* A bigger picture holds more detail but means more numbers to process, which is why the tool shrinks every frame to the same size before the model sees it.

A.4 Why this matters for your projects

- **The model never "sees" a three, a smile or a bottle.** It only sees patterns in numbers and learns which patterns usually go with which label.
- **Varied samples matter.** The same digit written thicker, tilted or in different light produces very different numbers. That is why Chapters 2, 7 and 8 ask for varied samples and why augmentation helps.
- **Light and background change the numbers too.** The same object in a dark room and in a bright room gives two different grids.

A.5 Try it

1. Draw a 5 × 5 grid on paper and shade squares to make a letter. Write the number for each square (0 or 255) and swap with a friend. Can they guess your letter from the numbers alone?
2. Open any photo on your computer and zoom in until you can see the squares. Roughly how many pixels wide is a thin line?

DRAFT

Appendix B

Sound, Waves and Spectrograms

For curious readers. Chapters 4, 5 and 9 use the word *spectrogram*. This appendix explains where it comes from. You can finish the course without it.

B.1 What is sound?

Sound is a **vibration of the air**. When you speak, your vocal cords open and close hundreds of times a second (Chapter 4), pushing the air back and forth. The moving air reaches a microphone, which turns the movement into an electric signal.

B.2 Reading a waveform

The picture a recorder draws is called a **waveform**: time goes left to right and the height shows how strong the vibration is at that moment.

- **Loudness** (volume) is the *height* of the wave: a loud sound makes tall peaks.
- **Pitch** (how high or low a sound is) depends on how *fast* the wave repeats: a high note vibrates fast and a low note vibrates slowly. The number of repeats per second is the **frequency**, measured in hertz (Hz).

Figure B.1 shows how the waveforms of "Yes" and "No" differ.

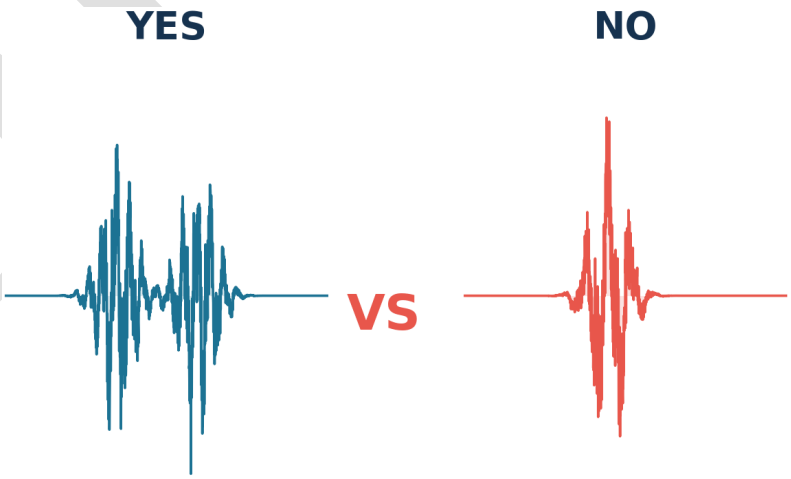


Figure B.1. The waveforms of "Yes" and "No" look different, which is what the model learns. *Image from the course slides by Rina Buoy, PhD.*

DRAFT

B.3 From sound to numbers

A computer measures the signal many times every second and stores each measurement as a number. Each measurement is a **sample**, and the number of samples per second is the **sample rate**. *(General background, not from the lessons: music CDs use 44,100 samples per second.)* A higher sample rate gives a more detailed recording and a bigger file. This is why file formats and clip length matter (Chapter 4).

B.4 Why a waveform is not enough

A waveform shows *how loud* the sound is over time, but a word is made of many pitches mixed together. Two different words can have similar loudness patterns. To tell them apart, it helps to see **which pitches are present at each moment**.

B.5 The spectrogram

A **spectrogram** is a picture of sound (as defined in Chapter 9):

- **left to right** is time,
- **bottom to top** goes from low pitch to high pitch,
- **brightness** shows how loud that pitch is at that moment.

Author's explanation of how it is made: the computer takes a very short slice of the sound, works out how much of each pitch is in it, and draws that as one thin vertical stripe. Putting the stripes side by side gives the picture.

Figure B.2 shows the spectrogram of the word "stop".

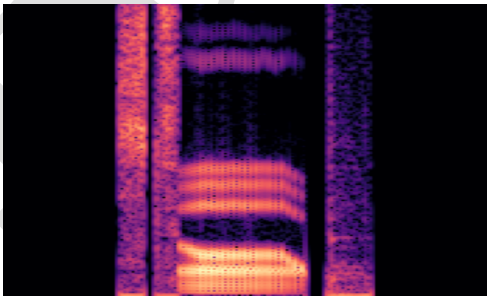


Figure B.2. Spectrogram of the word "stop". *Image from the course slides by Rina Buoy, PhD.*

B.6 Why spectrograms help AI

Once sound is a picture, a model that is good with pictures can learn from it. This is why audio classification in Teachable Machine is treated much like image classification: the word "yes" makes one kind of pattern and "no" makes another. It is also why **masking** (Chapter 9) works: blanking a vertical band hides one moment and blanking a horizontal band hides a range of pitches, so the model cannot depend on one small detail.

DRAFT

B.7 Try it

1. Use the course's Sound Wave Recorder (Chapter 5) to record "yes" in a low voice and then in a high voice. How does the waveform change?
2. Whistle a rising note. In which direction does the bright line move on a spectrogram?
3. Clap once. Is the picture a short stripe or a long band? Why?

DRAFT

Appendix C

How a Model Learns

For curious readers. Chapter 3 introduces epochs, learning rate and batch size without explaining the machinery. This appendix gives a gentle picture of what happens when you press **Train Model**. It is simplified on purpose and is the author's explanation, not taken from the slides.

C.1 Learning by guessing and correcting

Training is a loop of four steps:

1. **Guess.** The model looks at a training sample and guesses a label.
2. **Compare.** It checks the guess against the label you gave.
3. **Measure the mistake.** The size of the mistake is a number called the **loss**: a big loss means a bad guess.
4. **Adjust.** The model changes its inner settings a tiny bit so the same guess would be less wrong next time.

Then it repeats with the next samples, thousands of times.

C.2 What gets adjusted?

Inside the model are many numbers called **weights**. Each weight says how much attention to pay to one clue, such as an edge or a curve. Training nudges the weights up or down. You never touch them; Teachable Machine does it for you.

C.3 The three settings

Epochs. One epoch is one full pass through all your samples. More epochs give the model more chances to improve, but too many can make it memorise.

Batch size. Instead of adjusting after every single sample, the model looks at a small group (a **batch**) and adjusts once for the group. With the default batch size of 16 and 160 samples, one epoch has 10 adjustment steps ($160 \div 16$).

Learning rate. The learning rate sets how big each adjustment is. Imagine walking down a hill in fog to reach the valley, where the loss is smallest:

- Very **small** steps get you there, but very slowly.

DRAFT

- Very **large** steps make you jump over the valley again and again.
- A medium step, like the default **0.001**, usually works well.

C.4 Accuracy and error: right or wrong?

Before we talk about learning curves, we need two simple words. Both describe the same thing from opposite sides: *did the model get the picture right or wrong?*

- **Accuracy** is how often the model is **right**. If you show it 10 pictures and it names 8 of them correctly, its accuracy is 8 out of 10, or **80%**.
- **Error** is how often the model is **wrong**. In the same test it named 2 pictures wrongly, so its error is 2 out of 10, or **20%**.

Accuracy and error always add up to 100%, so a high accuracy means a low error:

accuracy + error = 100%

Everyday example: a quiz with 10 questions. Eight right answers is an 80% score (accuracy) and two wrong answers is 20% error.

For a picture model, "right" means the model's label matches the real label. Show a photo of a banana. If the model says "banana", that is a correct prediction; if it says "apple", that is a wrong one, and it counts toward the error. *(The terms "loss" in C.1 and "error" here are closely related: the loss is a more detailed way of measuring how wrong the guesses are, and training tries to make both smaller. This link is the author's simplification.)*

During training the model keeps trying to **reduce its error and raise its accuracy**. Chapter 3 shows this on a chart, and the next section explains why the curve eventually flattens. One more point, which matters in the rest of this appendix: there are two kinds of error.

- **Training error:** mistakes on the pictures the model learned from.
- **Error on new pictures:** mistakes on pictures it has never seen. This is the one that matters in real life.

C.5 Why accuracy levels off

When training works, the error falls quickly at first and then flattens (and accuracy rises and flattens), because the easy mistakes are fixed first. More epochs after that help little and can hurt.

Figure C.1 illustrates the idea with example curves.

C.6 Overfitting and underfitting

- **Underfitting:** the model has not learned enough yet. It does badly even on its own training samples.

DRAFT

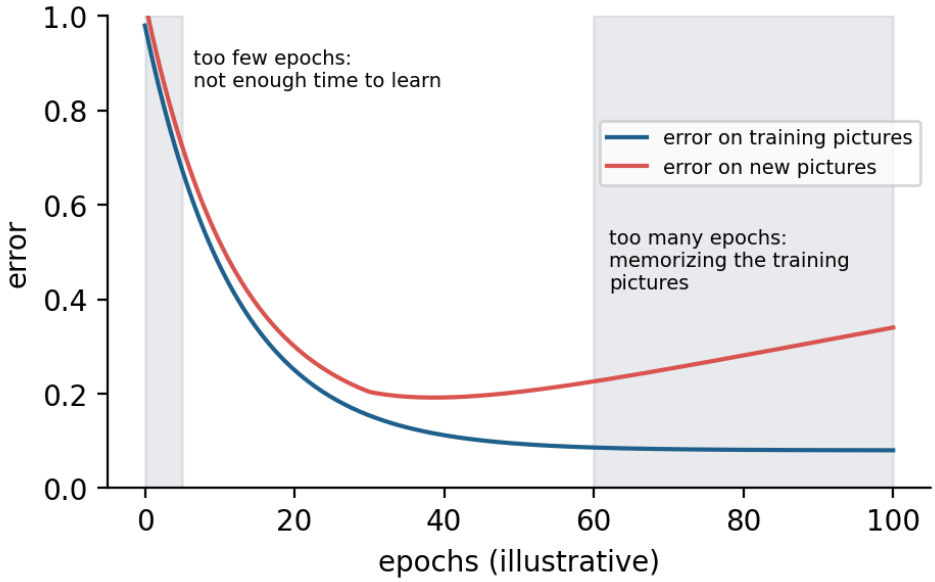


Figure C.1. An illustration of the epochs idea: too few epochs underfit and too many overfit (example curves, not measured data).

- **Overfitting:** the model has memorised the training samples, like a student who learned last year's answers but cannot solve new questions. It does very well on training samples and poorly on new ones.

This is why the course always tests with **new** samples (Chapters 3, 7 and 9), and why varied data and augmentation (Chapters 8 and 9) help: a model that has seen many variations has less to memorise.

C.7 What is a neural network?

Teachable Machine models are built from layers of simple units that pass numbers forward, loosely inspired by brain cells. Early layers pick out simple clues such as edges, curves or pitches; later layers combine them into bigger ideas such as a loop, an eye or a word. *(General background, not from the lessons.)* You do not design any of this. A ready-made model supplies most of it (see Appendix D).

C.8 Try it

1. Train the same model with 5, 20 and 50 epochs. Write down the accuracy each time. When does adding epochs stop helping?
2. Change only the learning rate, first very small, then very large. What happens to the accuracy?
3. A model is tested on 20 new pictures and names 17 correctly. What are its accuracy and its error?
4. In your own words, why is memorising a bad way to learn?

DRAFT

Appendix D

Pre-trained Models and Keypoints

For curious readers. Chapters 6 and 7 mention MobileNet, PoseNet and "17 keypoints". This appendix explains these ideas in more detail.

D.1 Why you need so few samples

Training a picture model from nothing would need a huge number of pictures and powerful computers. Teachable Machine avoids this with **transfer learning**:

1. A large model, such as **MobileNet** for pictures, is first trained on a very large collection of images. It learns general clues: edges, curves, textures and shapes.
2. Teachable Machine keeps that model and adds a small new part on top: **your classifier**.
3. Only your classifier is trained, with your samples. It learns which ready-made clues mean *your* classes.

That is why about 30–50 samples per class is enough to start (Chapters 6 and 7). *Author's analogy:* it is like learning a new alphabet when you already know how to read. You are not starting from zero.

Figure D.1 shows the transfer-learning pipeline for digits.

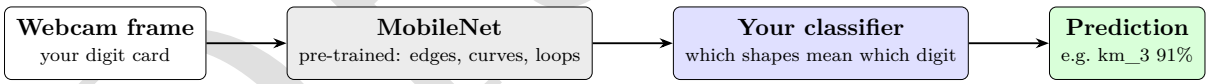


Figure D.1. Transfer learning for digits: MobileNet is pre-trained; only your classifier is trained.

D.2 Who does what

- **MobileNet** is the pre-trained picture model behind Image projects.
- **PoseNet** is the pre-trained body-position model behind Pose projects.
- **Your classifier** is the small part that belongs to you. It is the only part that changes when you train.

A borrowed model is only as good as what it already knows. PoseNet knows bodies, so it cannot tell an apple from a mango.

DRAFT

D.3 What is a keypoint?

A **keypoint** is a named point on the body, such as the nose, left shoulder or right knee. PoseNet finds **17 keypoints** (Chapter 6): five on the face, six on the arms and six on the hips and legs.

Figure D.2 shows the 17 keypoints on the body.

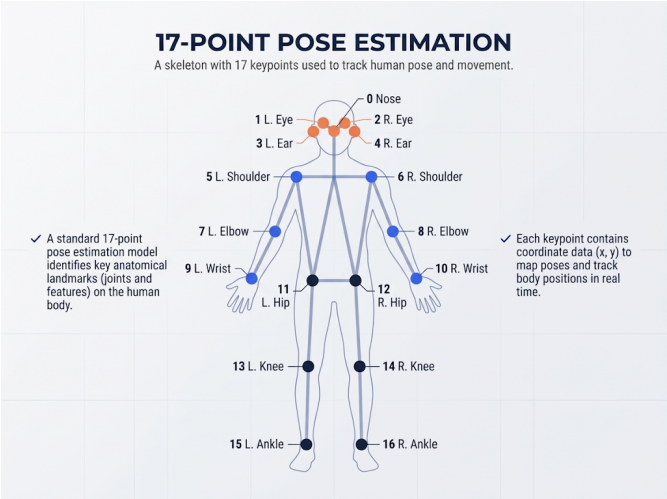


Figure D.2. 17-point pose estimation.

Each keypoint is stored as a pair of numbers **(x, y)**, its position in the picture. So a pose is a list of 17 pairs of numbers.

D.4 Why poses are easier than pictures

A camera frame has more than 150,000 numbers (Appendix A). A pose has only $17 \times 2 = \mathbf{34}$ **numbers**. The classifier has far less to sort through, and it ignores your clothes, the wall and the light. It only asks: *where are the body parts?* This is why a head tilt is easy to detect in Chapter 6: the two ear points are no longer level. *(Arithmetic and explanation are the author's.)*

D.5 Where pose models can go wrong

(General background, not from the lessons.)

- A body part is hidden or outside the camera view, so its keypoint is a guess.
- Poor light or very loose clothes make joints harder to find.
- Several people in the frame can confuse the model.

Check these first when a pose classifier misbehaves.

DRAFT

D.6 Try it

1. Stand in front of the pose preview and slowly tilt your head. Which keypoints move the most?
2. Cover your face with your hands. What happens to the skeleton and to the predicted class?
3. Why is a pose model a poor choice for telling an apple from a mango?

DRAFT

Appendix E

Measuring Results

For curious readers. Chapters 3, 7, 8 and 9 use total accuracy, class accuracy, confidence and the confusion matrix. This appendix walks through them with one small worked example. The numbers are made up by the author to show the method.

E.1 Accuracy

Accuracy is the share of test samples the model got right:

accuracy = correct answers ÷ total test samples

If a model gets 27 of 30 test clips right, its accuracy is 27 ÷ 30 = 0.9, or **90%**.

E.2 Why one number can hide problems

Imagine 30 test clips: 10 “yes”, 10 “no”, 10 “stop”. The model’s results:

Class	Tested	Correct	Class accuracy
yes	10	10	100%
no	10	10	100%
stop	10	7	70%

The **total accuracy** is 27 ÷ 30 = 90%, which sounds great, but “stop” is weak. **Class accuracy** (accuracy for each class on its own) shows where to add samples. This is why the course asks you to read both.

E.3 The confusion matrix

A **confusion matrix** is a table of *what the model said* against *what the sample really was*. As in Chapter 7, **rows are the real class** and **columns are the model’s guess**.

Real \ Guess	yes	no	stop
yes	10	0	0
no	0	10	0
stop	1	2	7

DRAFT

- The **diagonal** (10, 10, 7) holds the correct answers. Big numbers there are good.
- Numbers **off the diagonal** are mistakes. Here "stop" was guessed as "yes" once and as "no" twice.
- Add each row to get the number tested; the diagonal divided by the row sum is the class accuracy.

Figure E.1 shows the confusion matrix from the Chapter 8 example.

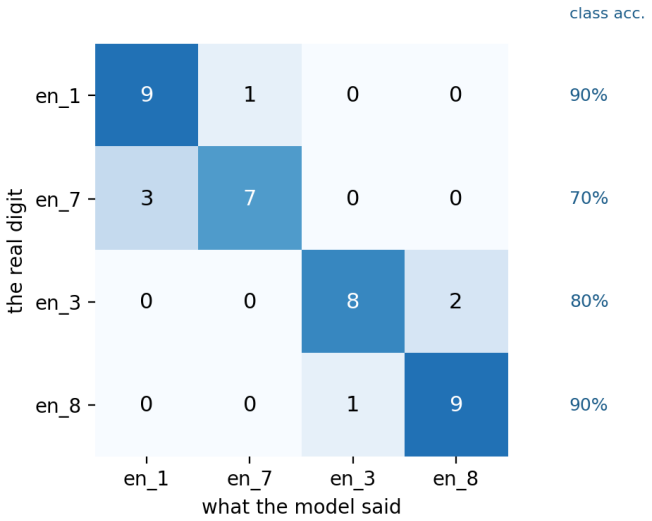


Figure E.1. The confusion matrix from the Chapter 8 example: rows are the real digit and columns what the model said.

How to use it: find the biggest off-diagonal number. That pair of classes is what to fix next, for example by adding more varied samples of both.

E.4 Confidence

For each prediction, Teachable Machine shows a **confidence**: a percentage for every class that adds up to 100%. A strong model puts high confidence on the right class; a weak one spreads it out (Chapter 5). A high confidence is not a promise: a model can be confidently wrong, especially on things it never saw in training.

Figure E.2 compares a strong and a weak model (example values).

E.5 Fair tests

Numbers are only trustworthy if the test is fair (Chapters 8 and 9):

- Test with **new** samples the model never trained on.
- Never augment test samples.
- Keep the test **balanced**: about the same number of samples in each class.
- Use the **same test** when comparing two models (Model A and Model B).
- Test with **different people, rooms and lighting**, not only yourself.

DRAFT

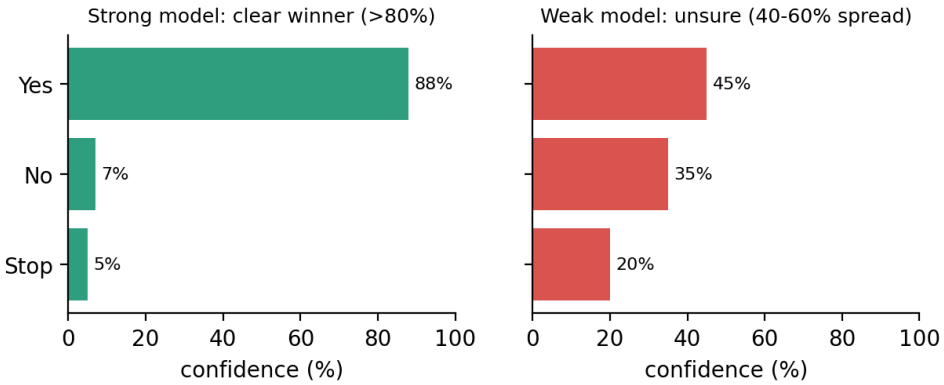


Figure E.2. A strong model puts high confidence on the right class; a weak model is unsure (example values).

E.6 Try it

1. A model scores 8 of 10 on “cat”, 10 of 10 on “dog” and 3 of 10 on “bird”. Work out the total accuracy and the class accuracies. Which class needs help?
2. Fill in a 3×3 confusion matrix for your own classifier. What is its biggest mistake?
3. Why is it unfair to test a model on the same samples it trained on?

DRAFT

About the Author



Dr. Rina Buoy (ប៊ួយ រីណា) is an AI researcher, engineer and teacher in Phnom Penh. She holds a PhD in Intelligent Informatics from Osaka Metropolitan University (2024). She is a Senior AI Researcher and Engineer at Techo Startup Center, Ministry of Economy and Finance, where she leads Khmer natural language processing and OCR work behind e-KYC services, and a Visiting Researcher at Osaka Metropolitan University.

Her research focuses on Khmer OCR and NLP, scene text recognition, document layout analysis, machine translation, and large language models and agents. She has published more than 20 peer-reviewed papers, including in IEEE Access, IJDAR and ICDAR, and is Principal Investigator on a JSPS KAKENHI grant on low-resource language document analysis (2026–2029).

Since 2020 she has taught AI, NLP, deep learning, reinforcement learning and computer vision as an adjunct lecturer at several Cambodian universities, and she runs AI training for businesses, professionals and secondary students through **RinAI**, an academy that trains Cambodians "from non-technical to technical". Her recognitions include Best Digital R&D of the Year (Cambodia Digital Awards 2022), a Teaching Excellence award from Paragon International University (2022) and the OxML 2023 Kaggle competition win.

No-code AI for Young Cambodian Learners grew out of her *Build Your Own AI* course for Grades 7–12.

Learn more: https://academy.rinabuoy.com

DRAFT